Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Генеративно-состязательные сети GAN для генерации изображений

Данная лабораторная работа посвящена решению задачи генерации изображений с помощью генеративно-состязательных сетей (GAN), используя библиотеку машинного обучения Keras.

Цель работы: получить навыки генерации изображений с помощью генеративно-состязательных сетей (GAN). В качестве обучающих данных используется набор данных CelebA с лицами людей.

Задачи:

1. Создание сети генератора.
2. Создание сети дискриминатора.
3. Обучение генаратора и дискриминатора в виде генеративно-состязательной сети (GAN).

Генеративно-состязательные сети (GAN)

Генеративно-состязательные сети (Generative Adversarial Networks, GAN), впервые представленные в 2014 году Яном Гудфеллоу и его коллегами, – альтернатива вариационным автокодировщикам для выделения скрытых пространств изображений. Они позволяют генерировать очень реалистичные искусственные изображения, статистически неотличимые от настоящих.

Чтобы проще было понять суть генеративно-состязательной сети, вообразите фальсификатора, пытающегося подделать картину Пикассо. Сначала он довольно плохо справляется с задачей. Он показывает свои подделки вместе с подлинниками Пикассо продавцу произведений искусства. Продавец оценивает подлинность картин и рассказывает фальсификатору, какие детали делают картину похожей на картину Пикассо. Фальсификатор возвращается в мастерскую и создает несколько новых подделок. С течением времени фальсификатор становится все более компетентным в имитации стиля Пикассо, а продавец – все более опытным в различении подделок. В конце концов у них на руках оказываются превосходные подделки Пикассо.

Вот что такое генеративно-состязательная сеть: она состоит из двух сетей – выполняющей подделку и оценивающей эту подделку, – постепенно обучающих друг друга:

- сеть-генератор – получает на входе случайный вектор (случайную точку в скрытом пространстве) и декодирует его в искусственное изображение;

- сеть-дискриминатор (или противник) – получает изображение (настоящее или поддельное) и определяет, взято ли это изображение из обучающего набора или сгенерировано сетью-генератором (рисунок 1).

Сеть-генератор обучается обманывать сеть-дискриминатор и, соответственно, учится создавать все более реалистичные изображения: поддельные изображения, почти неотличимые от настоящих. Сеть-дискриминатор, в свою очередь, постоянно адаптируется к увеличивающейся способности сети-генератора и устанавливает все более высокую планку реализма для генерируемых изображений. По окончании

обучения генератор способен превратить любую точку из своего входного пространства в правдоподобное изображение. В отличие от вариационных автокодировщиков это скрытое пространство дает меньше гарантий наличия в нем значимой структуры; в частности, оно не является непрерывным.

Примечательно, что генеративно-состязательная сеть (GAN) – это система, в которой минимум оптимизации не фиксирован. Обычно градиентный спуск заключается в постепенном скатывании вниз по холмам статического ландшафта потерь. Однако в случае с GAN каждый шаг вниз по склону немного меняет весь ландшафт. Это динамическая система, в которой процесс оптимизации стремится не к минимуму, а к равновесию двух сил. По этой причине генеративно-состязательные сети трудно поддаются обучению – чтобы получить действующую генеративно-состязательную сеть, требуется приложить большие усилия по настройке архитектуры модели и параметров обучения.

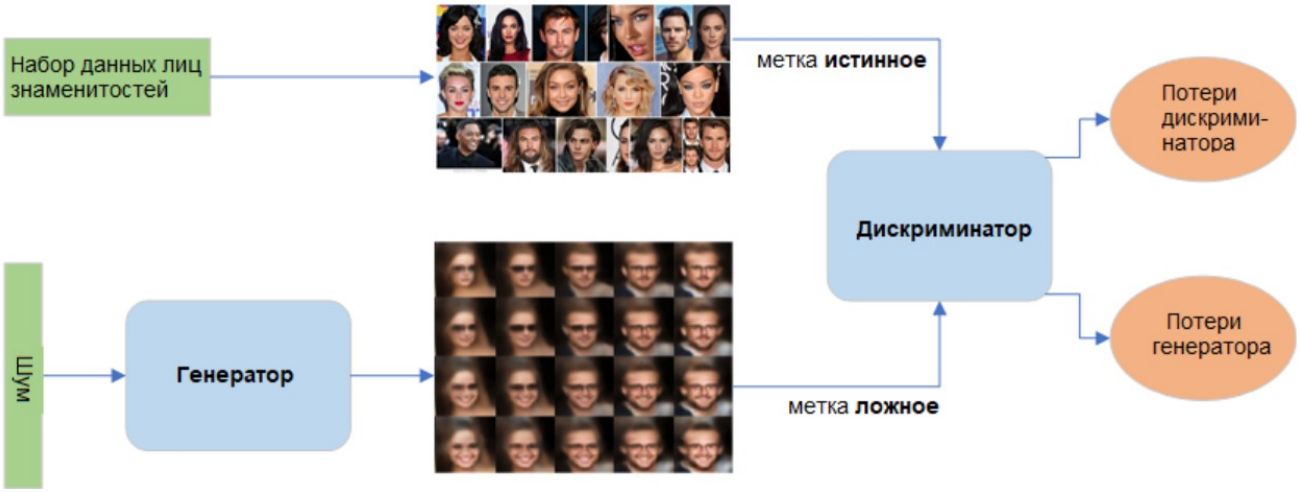


Рисунок 1. Схема работы GAN.

Обучающая выборка

Будем обучать GAN на изображениях из набора Large-scale CelebFaces Attributes (известного как CelebA), содержащего 200 000 изображений знаменитостей (http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html). Чтобы ускорить обучение, мы уменьшим размеры изображений до 64×64, то есть обучим сеть генерировать изображения человеческих лиц размером 64×64. В общих чертах GAN выглядит примерно так:

- сеть generator отображает векторы с формой (размерность_скрытого_пространства,) в изображения с формой (64, 64, 3);
- сеть discriminator отображает изображения с формой (64, 64, 3) в оценку вероятности, что изображение является настоящим;
- сеть gan объединяет генератор и дискриминатор gan(x) = discriminator(generator(x)), иными словами, отображает скрытое пространство векторов в оценку реализма этих скрытых векторов, декодированных генератором;
- обучим дискриминатор на примерах реальных и искусственных изображений, отмеченных метками «настоящее»/«поддельное», как самую обычную модель классификации изображений;
- для обучения генератора используем градиенты весов генератора в отношении потерь модели gan. То есть на каждом шаге мы будем смещать веса

генератора в направлении увеличения вероятности классификации дискриминатором изображений, декодированных генератором как «настоящие». Иными словами, мы будем обучать генератор обманывать дискриминатор.

Набор хитростей

Процесс обучения и настройки генеративно-состязательных сетей очень сложен. Однако есть несколько хитростей, которые следует знать и помнить. Как и многое другое в глубоком обучении, это больше алхимия, чем наука: все хитрости, описываемые далее, выявлены экспериментальным путем и не имеют теоретического обоснования. Они опираются на интуитивное понимание явления и хорошо работают на практике, хотя и не во всех контекстах. Вот несколько хитростей, используемых в реализации генератора и дискриминатора GAN. Это не полный список; еще множество хитростей, имеющих отношение к GAN, можно найти в специализированной литературе.

- В дискриминаторе вместо объединения мы будем уменьшать разрешение карт признаков за счет изменения шага выборки.

- Мы будем выбирать точки из скрытого пространства, используя нормальное распределение (распределение Гаусса), а не равномерное.

- Стохастичность повышает устойчивость. Поскольку целью обучения является динамическое равновесие, генеративно-состязательные сети легко могут застревать на разных препятствиях. Введение случайной составляющей в процесс обучения помогает предотвратить это. Мы будем вводить случайный компонент, добавляя случайный шум в метки для дискриминатора.

- Разреженные градиенты могут препятствовать обучению GAN. В глубоком обучении разреженность часто является желательным свойством, но не в случае с GAN. Разреженность градиента могут вызывать операции выбора максимального значения по соседним элементам (max pooling) и активации relu. Вместо выбора максимального значения для уменьшения разрешения я рекомендую увеличивать шаг свертки, а вместо функции активации relu использовать слой LeakyReLU. Он напоминает relu, но ослабляет ограничение разреженности, допуская небольшие отрицательные значения активации.

- В сгенерированных изображениях часто наблюдаются артефакты типа «шахматная доска», обусловленные неравномерным охватом пространства пикселей в генераторе. Для их устранения мы будем выбирать размер ядра, кратный размеру шага, при каждом использовании разреженных слоев Conv2DTranpose или Conv2D в генераторе и дискриминаторе.

Ход работы

1. Загрузка данных

Загрузить набор данных можно вручную с веб-сайта http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html. Если вы используете Google Colab, выполните следующие команды – они загрузят данные с Google Drive и распакуют их.

Создать рабочий каталог:

```
!mkdir celeba_gan
```

Загрузить архив с данными с помощью утилиты **gdown** (по умолчанию доступна в Colab; если нет – установите ее)

```
!gdown --id 1O7m1010EJjLE5QxLZiM9Fpjs7Oj6e684 -O celeba_gan/data.zip
```

Распаковать данные

```
!unzip -qq celeba_gan/data.zip -d celeba_gan
```

После распаковки изображений в каталог можно превратить каталог image_dataset_from_directory в объект набора данных. Так как нам нужны только изображения – без меток, – передадим аргумент label_mode=None.

```
from tensorflow import keras

dataset = keras.utils.image_dataset_from_directory("celeba_gan",
                                                   label_mode=None,
                                                   image_size=(64, 64),
                                                   batch_size=32,
                                                   smart_resize=True)
```

label_mode=None – возвращаться должны только изображения, без меток.

smart_resize=True – метокУменьшить размеры изображений до 64 × 64, используя комбинацию операций обрезки и масштабирования для сохранения пропорций. Нам не нужно, чтобы пропорции лиц искажались.

Наконец, масштабируем значения, представляющие пиксели, в диапазон [0–1].

```
dataset = dataset.map(lambda x: x / 255.)
```

Выведем на экран 8 случайных изображений из датасета CelebA (рисунок 2):

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 10))
for i, image in enumerate(dataset.take(8)):
    plt.subplot(4, 4, i+1)
    plt.axis("off")
    plt.imshow((image.numpy() * 255).astype("int32")[0])
plt.show()
```



Рисунок 2. Пример изображений из датасета CelebA

2. Дискриминатор

Теперь перейдем к модели дискриминатора, которая принимает на входе изображение-кандидат (реальное или искусственное) и относит его к одному из двух классов: «подделка» или «настоящее, имеющееся в обучающем наборе». Одна из многих проблем, часто возникающих в сетях GAN, – генератор создает изображения, которые выглядят как шум. Одно из возможных решений – использовать прореживание **Dropout** в дискриминаторе, что мы и сделаем здесь.

```
from tensorflow.keras import layers

discriminator = keras.Sequential(
.
.
Допишите код
.
.
```

3. Генератор

Затем реализуем модель generator, преобразующую вектор (из скрытого пространства – во время обучения он будет выбираться случайно) в изображение-кандидат. Скрытое пространство будет состоять из 128-мерных векторов.

```
latent_dim = 128
```

```
generator = keras.Sequential(
.
.
Допишите код
.
.
```

В первом слое **Dense** производится такое же количество коэффициентов (128), как в слое Flatten в кодировщике.

Слой **Reshape** восстанавливает слой **Flatten** кодировщика.

Слой **Conv2DTranspose** восстанавливает слой Conv2D кодировщика. После каждого такого слоя применяем **LeakyReLU** в качестве функции активации.

Выход сети будет иметь форму (28, 28, 1).

4. Генеративно-состязательная сеть

Наконец, перейдем к состязательной сети, объединяющей генератор и дискриминатор. В процессе обучения эта модель будет смещать веса генератора в направлении увеличения способности обмана дискриминатора. Эта модель преобразует точки скрытого пространства в классифицирующее решение – «подделка» или «настоящее» – и предназначена для обучения с метками, которые всегда говорят: «Это настоящие изображения». То есть обучение gan будет смещать веса в модели generator так, чтобы увеличить вероятность получить от дискриминатора ответ «настоящее», когда тот будет просматривать поддельное изображение.

Теперь можно приступать к обучению. Ниже схематически описывается общий цикл обучения. В каждой эпохе нужно выполнить следующие действия.

1. Извлечь случайные точки из скрытого пространства (случайный шум).
2. Создать изображения с помощью генератора, используя случайный шум.
3. Смешать сгенерированные изображения с настоящими.
4. Обучить дискриминатор на этом смешанном наборе изображений, добавив соответствующие цели: «настоящее» (для настоящих изображений) или «подделка» (для сгенерированных изображений).
5. Выбрать новые случайные точки из скрытого пространства.
6. Обучить gan, используя эти случайные векторы, с целями, которые всегда говорят: «Это настоящие изображения». Данное действие приведет к смещению весов генератора (и только генератора, потому что внутри gan дискриминатор «замораживается») в направлении, увеличивающем вероятность получить от дискриминатора ответ «настоящее» для сгенерированных изображений: это научит генератор обманывать дискриминатор.

Определим подкласс класса Model с нестандартным методом **train_step()**. На этот раз мы будем использовать два оптимизатора (один для генератора и один для дискриминатора), поэтому мы также переопределим метод **compile()**, чтобы реализовать возможность передачи двух оптимизаторов.

```
import tensorflow as tf

class GAN(keras.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super().__init__()
        self.discriminator = discriminator
        self.generator = generator
        self.latent_dim = latent_dim
```

Настройка метрик для отслеживания двух потерь в каждой эпохе обучения:

```
        self.d_loss_metric = keras.metrics.Mean(name="d_loss")
        self.g_loss_metric = keras.metrics.Mean(name="g_loss")
```

```
    def compile(self, d_optimizer, g_optimizer, loss_fn):
```

```
        super(GAN, self).compile()
        self.d_optimizer = d_optimizer
        self.g_optimizer = g_optimizer
        self.loss_fn = loss_fn
```

Настройка метрик для отслеживания двух потерь в каждой эпохе обучения:

```
    @property
    def metrics(self):
        return [self.d_loss_metric, self.g_loss_metric]
```

Выбор случайных точек из скрытого пространства:

```
    def train_step(self, real_images):
            batch_size = tf.shape(real_images)[0]
            random_latent_vectors = tf.random.normal( shape=(batch_size,
self.latent_dim))
```

Декодирование точек в поддельные изображения:

```
            generated_images = self.generator(random_latent_vectors)
```

Объединение поддельных изображений с настоящими:

```
            combined_images = tf.concat([generated_images, real_images],
axis=0)
```

Сбор меток, отличающих настоящие изображения от поддельных

```
            labels = tf.concat([tf.ones((batch_size, 1)),
tf.zeros((batch_size, 1))], axis=0)
```

Добавление случайного шума в метки – это важный шаг!

```
            labels += 0.05 * tf.random.uniform(tf.shape(labels))
```

5. Обучение дискриминатора

Для обучения дискриминатора вставьте следующий код:

```
        with tf.GradientTape() as tape:
            predictions = self.discriminator(combined_images)
            d_loss = self.loss_fn(labels, predictions)
        grads = tape.gradient(d_loss, self.discriminator.trainable_weights)
        self.d_optimizer.apply_gradients( zip(grads,
self.discriminator.trainable_weights) )
```

Выбор случайных точек из скрытого пространства:

```
        random_latent_vectors = tf.random.normal(shape=(batch_size,
self.latent_dim))
```

Сбор меток, говорящих: «Все эти изображения настоящие» (это ложь!):

```
        misleading_labels = tf.zeros((batch_size, 1))
```

6. Обучение генератора

Для обучения генератора вставьте следующий код:

```
        with tf.GradientTape() as tape:
            predictions =
self.discriminator(self.generator(random_latent_vectors))
            g_loss = self.loss_fn(misleading_labels, predictions)
        grads = tape.gradient(g_loss, self.generator.trainable_weights)
        self.g_optimizer.apply_gradients(zip(grads,
self.generator.trainable_weights))
```

```
        self.d_loss_metric.update_state(d_loss)
        self.g_loss_metric.update_state(g_loss)
        return {"d_loss": self.d_loss_metric.result(), "g_loss":
self.g_loss_metric.result()}
```

Перед началом обучения настроим обратный вызов для отслеживания результатов: он будет с помощью генератора создавать и сохранять несколько поддельных изображений в конце каждой эпохи.

```
class GANMonitor(keras.callbacks.Callback):
    def __init__(self, num_img=3, latent_dim=128):
        self.num_img = num_img
        self.latent_dim = latent_dim

    def on_epoch_end(self, epoch, logs=None):
        random_latent_vectors = tf.random.normal(shape=(self.num_img,
self.latent_dim))
        generated_images = self.model.generator(random_latent_vectors)
        generated_images *= 255
        generated_images.numpy()
        for i in range(self.num_img):
            img = keras.utils.array_to_img(generated_images[i])
            img.save(f"generated_img_{epoch:03d}_{i}.png")
```

7. Обучение генеративно-состязательной сети

Для обучения генеративно-состязательной сети вставьте следующий код:

```
gan = GAN(discriminator=discriminator, generator=generator,
latent_dim=latent_dim)

gan.compile(d_optimizer=keras.optimizers.Adam(learning_rate=0.0001),
            g_optimizer=keras.optimizers.Adam(learning_rate=0.0001),
            loss_fn=keras.losses.BinaryCrossentropy(),)
```

Поставим 100 эпох обучения. Интересные результаты начнут появляться примерно после 20-й эпохи.

```
gan.fit(dataset, epochs=100, callbacks=[GANMonitor(num_img=10,
latent_dim=latent_dim)])
```

Процесс бучения сетей GAN очень длительный, возможно вам потребуется около 2-3 часов, чтобы увидеть первые более–менее приемлемые результаты генерации. Весь процесс обучения может достигать нескольких дней.

8. Результат работы нейронной сети

После завершения каждой эпохи обучения в левой панели Google Colab будут отображаться сгенерированные изображения. Можете посмотреть динамику их изменения. На рисунке 3 показаны примеры изображений, которые сеть GAN смогла сгенерировать после 30 эпох обучения. Как видно на изображения присутствуют некоторые артефакты и искажения, который будут постепенно устраняться при продолжении обучения.

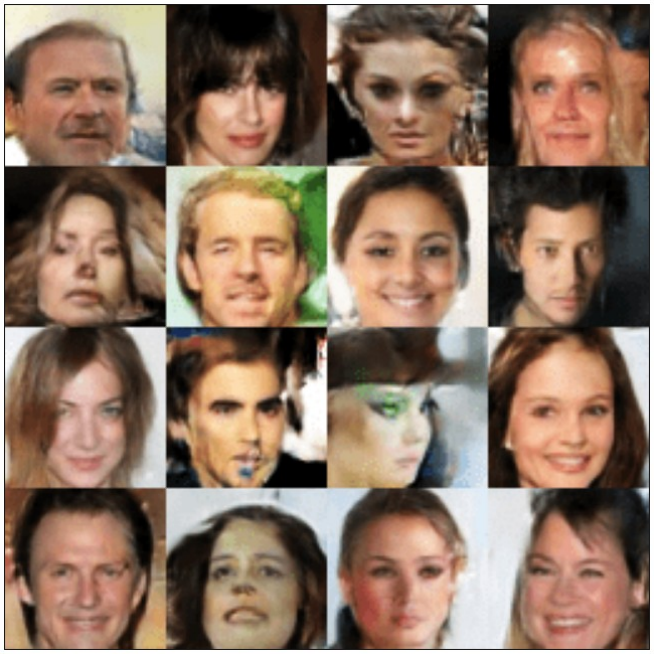


Рисунок 3. Несколько изображений, сгенерированных после 30 эпох обучения

В ходе обучения можно заметить, что потери состязательной сети значительно возрастают, а потери дискриминатора стремятся к нулю – дискриминатор может оказаться в доминирующем положении над генератором. В этом случае попробуйте уменьшить скорость обучения дискриминатора и увеличить его коэффициент прореживания.

Вывод

В результате выполнения данной лабораторной работы Вы научились решать задачу генерации изображений с помощью генеративно-состязательных сетей GAN, используя библиотеку машинного обучения Keras.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания - 0 баллов.

Большей частью выполнено - 2 балл.

Выполнено полностью - 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету - 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований - 2 балл.

Соответствует всем требованиям - 4 балла.